Beyond Destinations: Instruction-Aware Graph Path Planning and Navigation with OpenStreetMap

Anonymous Authors

***Abstract*— We propose BD-Nav, a novel instruction-aware navigation framework that integrates large language models with OpenStreetMap graphs to generate routes that align with user intent. Unlike prior approaches that only extract destinations from language, BD-Nav interprets free-form instructions to identify not only subgoals but also intermediate routes that satisfy conditions such as safety, weather, or avoidance constraints. Our system employs sequential large language model modules for intent parsing, weight function generation, and path evaluation, enabling iterative refinement of graph-based routes. We validate BD-Nav on real-world outdoor experiments with a quadruped robot, demonstrating its ability to follow user instructions without pre-built maps. Results show improved success rates in multi-subgoal scenarios and enhanced robustness in incomplete OpenStreetMap regions compared to ablation variants. This work highlights the potential of combining open-source maps and large language model reasoning to achieve practical, user-centered robot navigation. Code and supplementary materials are available at https://bd-nav.github.io/.**

I. Introduction

As autonomous navigation technology advances, robots such as guide and service robots are becoming increasingly prevalent in everyday environments [1]. To operate effectively alongside humans, these robots must understand natural language commands and navigate accordingly [2]. While traditional interfaces, such as buttons [3] and mobile applications [4], perform reliably in structured and well-mapped settings, they are less suitable in outdoor or unfamiliar environments where maps may be incomplete or unavailable. In such scenarios, robots must interpret free-form natural language instructions and generate navigation plans that reflect user intent [5]. Notably, user instructions often include not only explicit destinations but also implicit preferences, such as safety requirements, environmental context, or route-related constraints [6], motivating navigation beyond conventional shortest-path planning.

Existing research on language-based spatial understanding and navigation [7]–[14] can be broadly categorized by their primary focus. Some studies [7], [8] concentrate on language-based place recognition, providing semantic grounding for localization but not full route generation or motion execution. Other works [9] incorporate simple linguistic cues, such as "narrow path" or "wide path," into occupancy grid-based costmaps. Although these approaches demonstrate the feasibility of language-guided planning, they remain limited in linguistic expressiveness and often lack validation through real-world navigation experiments.

More recent approaches directly address natural language-based navigation. WanderGuide [11] extracts a destination from user instructions and matches it to a pre-built database of visually annotated landmarks, while E2Map [12] leverages visual-language maps to support sequential multi-destination navigation. LM-Nav [13] and OpenBench [14] integrate large models with topological or OSM-based representations to improve scalability in real-world environments. Despite these advances, most existing methods primarily emphasize identifying and reaching destinations. They do not explicitly model route-level constraints or ensure that intermediate paths reflect user preferences throughout the entire journey.

To address these limitations, as illustrated in Fig. 1, we propose BD-Nav, an instruction-aware navigation framework that grounds natural language instructions in OpenStreetMap (OSM) graphs. Unlike prior approaches that focus mainly on destination extraction, BD-Nav interprets user instructions to determine both desired destinations and corresponding intermediate route conditions. The framework employs LLM-based modules to generate instruction-aligned graph paths, presents the planned route for verification, and allows iterative refinement to better match user intent. The validated graph path is then integrated into a rule-based quadruped navigation framework, enabling stable and reliable real-world navigation across multiple destinations.

The contributions of BD-Nav are summarized as follows:

- We propose an LLM-based framework that generates instruction-aligned graph paths by grounding natural language commands in OSM graphs, identifying both subgoals and intermediate route constraints.
- We introduce a Path Evaluator that verifies and refines generated routes to better match the intended navigation.
- We integrate the graph-based planner into a practical navigation framework and validate BD-Nav through real-world quadruped robot experiments.

II. Related Works

A. Pose-Goal Navigation

Traditional pose-goal navigation approaches [15], [16] provide a numeric goal pose to the robot and compute a path on a pre-built occupancy grid map. Because the goal is explicitly specified in metric coordinates, these methods minimize ambiguity in destination interpretation and allow navigation to begin immediately. They perform reliably in structured environments where prior maps and accurate localization are available. However, the requirement for precise goal poses limits their flexibility in open-world or outdoor scenarios, where users may prefer to express navigation objectives in natural language rather than exact coordinates.

B. Landmark-based Navigation

Landmark-based navigation methods [11], [17], [18] represent goals using semantic descriptors such as text or im-

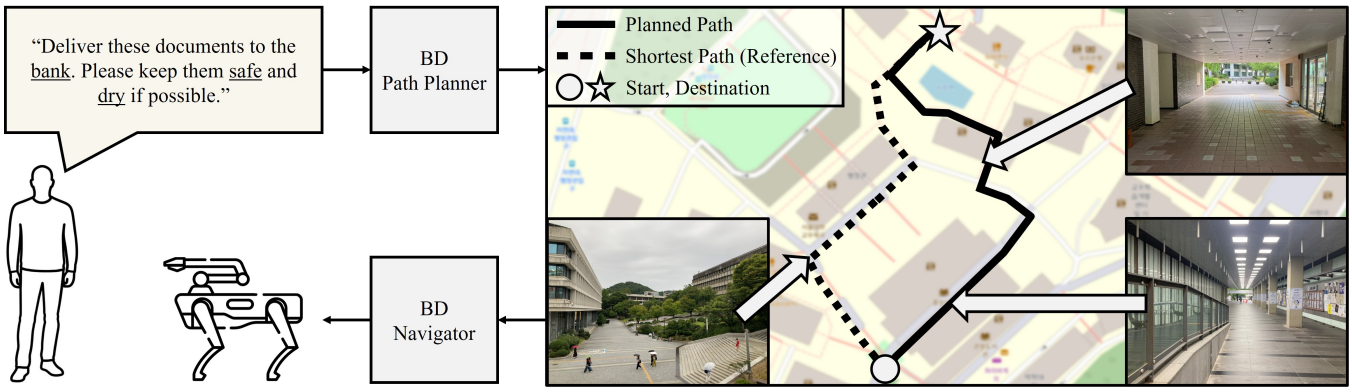


Fig. 1. BD-Nav concept scenario. The robot navigates primarily through buildings to deliver documents to the destination while avoiding the rain, following the user's instructions.

ages, improving usability compared to pose-based interfaces. ViNG [17] constructs a topological graph from visual observations and matches current and goal images to graph nodes for shortest-path planning. DRAGON [18] enables user-defined visual landmarks within a SLAM-generated map and selects destinations using text–image similarity. WanderGuide [11] similarly associates text labels with mapped landmarks for navigation. While these approaches remove the need for explicit numeric goals, they typically support single-destination navigation and emphasize shortest-path selection, without accounting for route-level constraints or sequential multi-goal planning.

C. Language-based Navigation

As shown in Table I, language-based navigation studies [12]–[14] interpret free-form natural language instructions to extract destination information and generate navigation plans accordingly. Because multiple destinations can be expressed within a single sentence, these methods support sequential navigation across subgoals. Additional linguistic cues, such as relative directions or distances, can further refine goal selection. Nevertheless, similar to landmark-based approaches, most existing methods focus primarily on identifying destinations rather than shaping the intermediate routes between them. As a result, they cannot fully capture user intent when route conditions, such as safety preferences, environmental context, or avoidance constraints, must be satisfied throughout the entire journey.

Unlike prior work, BD-Nav grounds natural language in OpenStreetMap graphs to perform instruction-aware planning, modeling both destination goals and intermediate route constraints, thus allowing user intent to shape not only where the robot goes, but also how it travels between subgoals.

III. Methods

A. Problem Definition

BD-Nav enables a quadruped robot to navigate to out-of-sight outdoor destinations using free-form natural language. It interprets instructions to determine both target locations and route-level constraints, jointly modeling graph feasibility and semantic preferences.

As shown in Fig. 2, given a user instruction T and OSM graph G_0, the planner generates an instruction-aligned path P, which is executed by the navigation module using sensor data S. The overall system is expressed as:

$$P = f_{\text{plan}}(T, G_0), \tag{1}$$

$$U = f_{\text{nav}}(P, S), \tag{2}$$

where T is the user instruction, G_0 the OSM graph, P the generated graph path, S the sensor observations, and U the robot control input. The function f_{plan} produces a graph path from language and map data, and f_{nav} converts it into executable commands using real-time perception and localization.

TABLE I

Related work on language-based navigation.

	Multiple Goals	w/o Prior Map	Intermediate Route
[12], [13]	✓		
[14]	✓	✓	
Ours	✓	✓	✓

B. BD Path Planner

As depicted in Fig. 3, BD-Nav is formulated as a language-conditioned constrained shortest path problem on an OSM-derived graph. Rather than treating natural language as a mere destination selector, the planner interprets instructions as specifying both structural constraints and preference-induced metric deformation. This formulation unifies goal satisfaction, waypoint ordering, avoidance constraints, and semantic cost shaping within a single graph optimization framework.

As shown in Fig. 4, a local OSM subgraph is extracted as

$$G = (V, E) = f_{\text{graph}}(G_0, p, \delta), \tag{3}$$

where G denotes the loaded graph, V the set of nodes, and E the set of directed edges representing traversable road segments. The variable $p \in V$ denotes the robot's current position projected onto the graph, and $\delta > 0$ denotes the spatial radius used to extract a region of interest from the

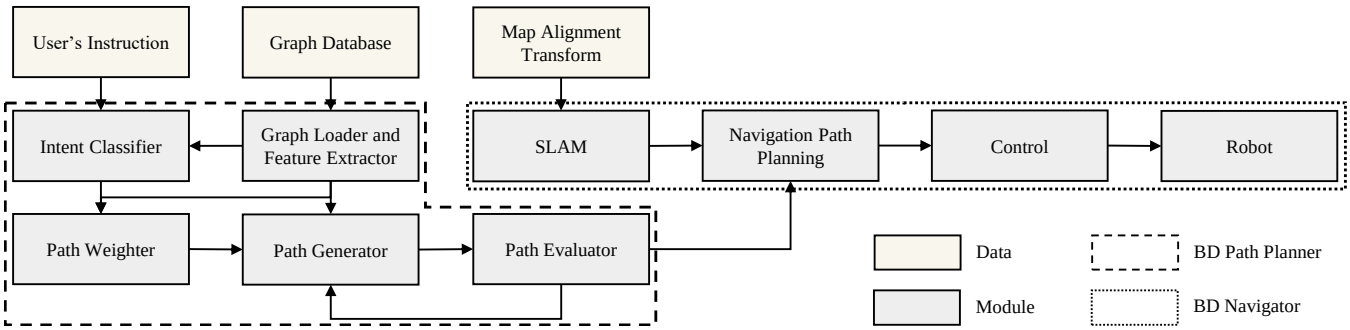


Fig. 2. System architecture. The system generates an OSM-based graph path from the user's instruction and uses a rule-based navigation framework to follow the generated path.

global database G_0. The function f_{graph} restricts planning to a spatially bounded local neighborhood.

Each edge $e \in E$ is associated with geometric attributes such as length and connectivity, as well as semantic attributes derived from OSM tags, including surface type, incline, tunnel presence, and environmental metadata. A structured feature representation is computed as

$$F = f_{\text{feat}}(G), \tag{4}$$

where F denotes extracted node- and edge-level descriptors and f_{feat} the feature extraction process. These features provide the semantic grounding necessary for language interpretation and cost construction.

1) Intent Classifier

The Intent Classifier processes the natural language instruction T together with graph features F using a large language model, extracting structured planning elements:

$$\{D, W, A, C\} = f_{\text{intent}}(T, F), \tag{5}$$

where $D \subseteq V$ denotes candidate destination nodes, $W = (w_1, \ldots, w_m)$ ordered waypoint nodes, A avoidance constraints over nodes or edges, and C route-level preference conditions. The function f_{intent} grounds free-form text into graph-compatible symbolic representations.

These components define the structural specification of the navigation task. The admissible path set is defined as

$$\mathcal{Q}(G, p, D, W, A),$$

where $\mathcal{Q}$ consists of all paths in G that start at p, reach at least one node in D, respect waypoint ordering, and avoid restricted elements.

A path $Q \in \mathcal{Q}$ is represented as an ordered sequence of connected edges

$$Q = (e_1, \ldots, e_K), \quad e_i \in E,$$

such that consecutive edges share incident nodes. Hard constraints are enforced by restricting $\mathcal{Q}$ to exclude prohibited edges or invalid waypoint orderings.

2) Path Weighter

While D, W, and A determine structural feasibility, the route-level constraints C influence preference among admissible paths. The Path Weighter constructs a language-conditioned edge cost function

$$\Psi = f_{\text{weight}}(F, C), \quad \Psi : E \to \mathbb{R}_{\geq 0}, \tag{6}$$

where $\Psi(e)$ denotes the traversal cost assigned to edge e. The function f_{weight} synthesizes this cost from graph features and route-level conditions, and the codomain $\mathbb{R}_{\geq 0}$ denotes nonnegative real numbers.

This induces a deformation of the underlying graph metric. Unlike classical shortest-path planning where cost is proportional to geometric length, BD-Nav augments geometric distance with language-conditioned semantic terms. For example, sheltered edges may receive reduced cost under weather-related preferences, while steep or unsafe edges incur higher penalties under safety-oriented instructions. Hard avoidance constraints can equivalently be implemented by assigning infinite cost to prohibited edges, thereby unifying feasible-set restriction and cost penalization within a single optimization formulation.

3) Path Generator

The instruction-aligned graph path is computed as

$$P^* = \underset{Q \in \mathcal{Q}(G,p,D,W,A)}{\arg\min} \sum_{e \in Q} \Psi(e), \tag{7}$$

where P^* denotes the optimal path under the language-conditioned metric and $\sum_{e \in Q} \Psi(e)$ the cumulative traversal cost. The operator $\arg\min$ selects the admissible path with minimum total cost.

In practice, this optimization is realized via

$$P = f_{\text{search}}(G, D, W, A, p, \Psi), \tag{8}$$

where f_{search} denotes A* [19] search operating on graph G with structural constraints and cost function Ψ. The returned path P approximates the optimal solution P^* under the discrete graph representation.

4) Path Evaluator

After path generation, a single-step semantic verification is performed. The graph path is projected onto a visual representation

$$I = f_{\text{image}}(G, P), \tag{9}$$

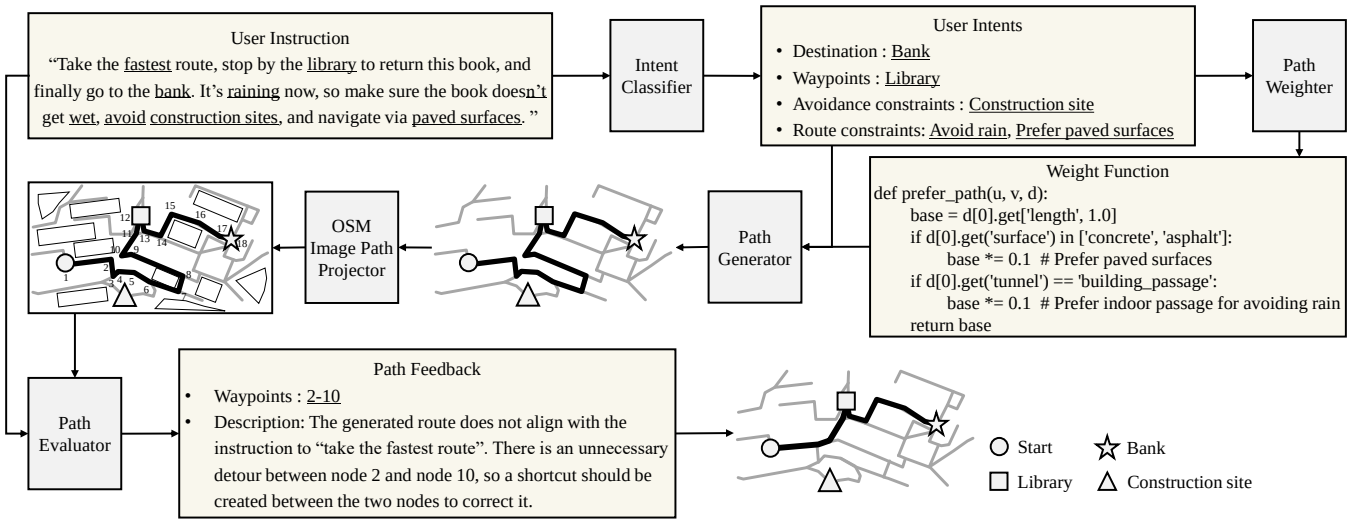


Fig. 3. Path planning process from the Intent Classifier to the Path Evaluator. The system receives a user instruction as input, identifies destination and waypoint navigation intents, and forwards them to the Path Weighter module, which generates weight function code for OSM graph path planning.

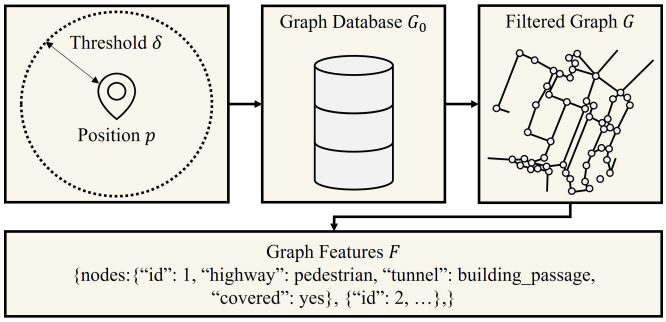


Fig. 4. Process of graph loading and feature extraction. A distance-filtered OSM subgraph is loaded based on the initial position and threshold, and node and edge labels and attributes are extracted to construct graph features.

where I denotes the image of path P overlaid on the OSM map. This provides spatial context suitable for multimodal reasoning.

A refinement operator is then applied:

$$\hat{P} = f_{\text{ref}}(P; T), \tag{10}$$

where $\hat{P}$ denotes the refined path and f_{ref} maps a candidate path and instruction to a corrected path within the admissible set $\mathcal{Q}(G, p, D, W, A)$. If no inconsistency is detected, $\hat{P} = P$, and the refined path is forwarded to the navigation module.

From an optimization perspective, this stage can be interpreted as a single-step projection toward the instruction-consistent subset of $\mathcal{Q}$. The initial solution P may imperfectly capture high-level semantic intent due to abstraction or cost approximation. The refinement step adjusts the solution toward improved semantic consistency while remaining within the feasible graph domain. In the current framework, this verification–correction mechanism is executed only once and does not constitute an iterative descent process.

C. BD Navigator

The BD Navigator executes the validated graph path on the quadruped robot. LiDAR-based SLAM constructs an occupancy grid map and estimates odometry in real time. The OSM-derived graph path is aligned with the SLAM map using GPS and IMU information or user-provided initialization when required.

For subgoals within the mapped region, A* planning is performed directly in free space. For subgoals outside the current map, a reachable intermediate goal is selected within free space and connected toward the intended subgoal direction. The final trajectory is tracked using Model Predictive Control (MPC) [20], which computes velocity commands producing the control input U applied to the robot.

IV. Experiments

We evaluate BD-Nav along four complementary axes: (A1) instruction alignment, (A2) the contribution of OSM features and the Path Evaluator, (A3) robustness across diverse navigation scenarios, and (A4) real-world applicability.

A. Experimental Setup

1) Metrics and Data

We evaluate BD-Nav using precision, recall, and F1 score, which are commonly adopted in discrete node-based graph path planning tasks on OSM graphs. Ground-truth graph paths corresponding to user instructions are manually generated from OSM data. The experimental scenarios include, for example, specific region and terrain selection with avoidance constraints, shortest-path navigation, and selecting the closest destination among multiple identical goal candidates. For each scenario, navigation commands are designed based on OSM data, and the corresponding reference paths are manually annotated. The predicted paths are then compared with the ground-truth paths at the OSM node level to compute the evaluation metrics. As shown in Fig. 5, a true positive (TP) is defined as a predicted node that appears in the ground-truth path, while false positives (FP) and false negatives (FN)

TABLE II

USER INSTRUCTION SCENARIO CATEGORIES FOR BD-NAV.

Scenario	Description	Example
Mobility & Accessibility	Accessibility-constrained routing under physical limitations.	"Navigate to the post office while avoiding stairs and steep slopes, since the robot is carrying a load and must remain on flat, paved roads."
Environmental & Situational	Context-aware routing under environmental conditions.	"Go to the bank using well-lit and populated streets because it is nighttime and visibility is limited."
Complex Routing	Multi-step routing with ordered waypoints or task constraints.	"First stop by the convenience store to pick up a package, then proceed to the bank, ensuring the stops are visited in that order without unnecessary detours."
Safety & Hazard Avoidance	Safety-aware routing that avoids hazardous areas.	"Travel to the cafe while avoiding construction sites and railway crossings, prioritizing pedestrian-friendly paths even if the route becomes slightly longer."

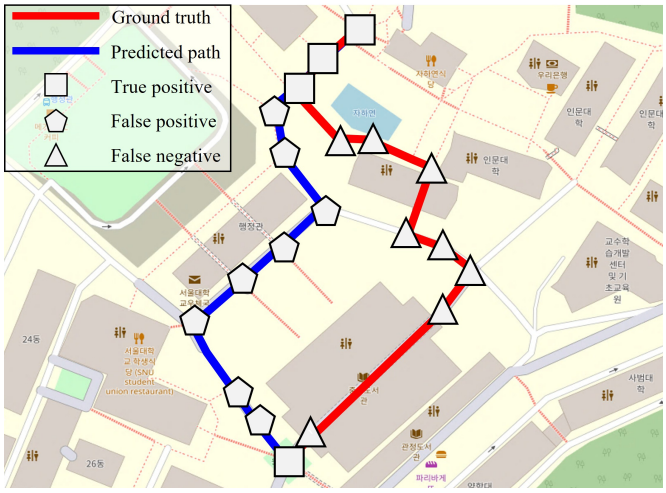


Fig. 5. Node-level comparison between ground-truth and predicted paths, illustrating TP, FP, and FN classifications.

correspond to unnecessary and missing nodes, respectively. Both predicted and ground-truth nodes are defined on the same OSM graph, and only exact node matches are considered correct, excluding near-miss cases. Under this definition, high precision reflects fewer unnecessary nodes, high recall indicates better coverage of the ground-truth path, and the F1 score provides a balanced measure of the two.

2) Environments and Scenarios

To evaluate the BD Path Planner, experiments were conducted across one university campus and four global locations in urban and suburban environments selected based on established criteria [21]. Multiple path planning scenarios were designed for each environment. In addition, two campus scenarios derived from the generated OSM graph paths were used to validate the performance of the BD Navigator.

As shown in Table II, we classify navigation instructions into four types derived from classical graph-based and constrained routing formulations [19], [22], [23], namely mobility- and accessibility-constrained routing, environmental and situational routing, multi-stop routing with ordered waypoints, and safety- and hazard-avoidance routing. We evaluate all categories to verify that the proposed framework produces routes aligned with diverse user intents.

3) Implementation Details

For path planning evaluation, GPT [24] and Gemini [25] were used to compare LLM- and VLM-based approaches. The OSM loading radius δ was set to 200 m, and the temperature of both models was fixed to 0 for reproducibility. The MPC prediction horizon was 10 steps. Real-world experiments were conducted on a Unitree Go1 quadruped equipped with a Velodyne VLP-16 LiDAR, Intel NUC, Xsens MTi-680G IMU, and u-blox GNSS, with ROS2 [26] was used to integrate all system modules.

B. Evaluation

1) Path Planning

The experimental results for the BD Path Planner are presented in Fig. 6, Table III, and Table IV. Figs. 6(a) and 6(b) correspond to campus experiments, while Figs. 6(c) and 6(d) illustrate urban and suburban environments. In Fig. 6(a), the BD Path Planner generates a safe route that avoids stairs and primarily follows flat terrain, even though it is longer than the shortest path, demonstrating its ability to reflect safety-related user constraints. In Fig. 6(c), the planner correctly produces a route that sequentially visits subgoals such as nearby cafes, rather than moving directly to the final destination. Similarly, in Fig. 6(d), while the shortest path follows main roads, the BD Path Planner successfully generates a route that prioritizes residential roads as instructed by the user. An interesting observation is that when the direct path to the target church was missing in the OSM data, the closest available OSM node was set as the destination, which led to an extra segment being generated beyond the church. We plan to address such cases in future work by integrating the Path Evaluator for automatic correction. Fig. 6(b) highlights the role of the Path Evaluator. While the shortest path bypasses a region due to missing OSM roads, the Path Evaluator predicts a connection between nearby nodes, shown as red points, creating a more direct route that reflects real roads and validates A1 and A2. However, as shown in Fig. 7, the Path Evaluator may occasionally generate paths that do not exist in the actual environment due to its reliance on commonsense reasoning. For example, given the instruction "Go to the bank using the fastest route," the feedback "connect nodes 8 and 10" results in the shortest path between the two nodes being predicted, even though they are not connected in the underlying graph.

Table III compares the performance of the BD Path

TABLE III

PATH PLANNING ACCURACY BY MODEL AND DATA AVAILABILITY. (d_{H}: HAUSDORFF DISTANCE, l_{C}: PATH LENGTH RATIO.)

Case	# of Scenarios	LLM	VLM	with OSM graph features	d_{H} (m)	l_{c}	F1 (%)
1	14	GPT-4	GPT-4o	✓			**94.87**
2		GPT-4o	GPT-4o	✓			71.25
3		Gemini-2.5-flash	GPT-4o	✓			90.07
4	11	GPT-4	-	✓			54.32
5		GPT-4	GPT-4-Turbo	✓			**88.86**
6		GPT-4	GPT-4o	✓			86.60
7	9	GPT-4	GPT-4o	-			65.79
8		GPT-4	GPT-4o	✓			**96.00**

TABLE IV

PATH PLANNING ACCURACY BY ENVIRONMENT.

Area	# of Scenarios	Environment	d_{H} (m)	l_{c}	F1 (%)
1	7	Urban			95.77
2	9	Urban			96.68
3	10	Suburban			90.60
4	10	Suburban			93.21

TABLE V

GRAPH SIZE AND PATH PLANNING LATENCY BY REGION AND RADIUS. (δ: RADIUS, n: NODE COUNT, t: LATENCY, $m1$: INTENT CLASSIFIER, $m2$: PATH WEIGHTER, $m3$: PATH GENERATOR, $m4$: PATH EVALUATOR)

		t (s)				
δ (m)	n	$m1$	$m2$	$m3$	$m4$	total
100	114	1.22	4.12	0.01	7.96	14.08
500	1,339	1.78	4.82	0.09	8.31	16.87
1,000	5,794	2.73	8.87	1.64	8.20	26.68

Planner under different configurations. GPT-4 achieved the best results among the tested LLMs and was therefore adopted in later experiments. For VLMs, adding the Path Evaluator improved performance by approximately 61.5% in terms of F1 score compared to the case without a VLM, highlighting its critical role and further validating A2. GPT-4-Turbo performed slightly better at correcting erroneous paths, however it was less decisive when validating correct ones, which resulted in only a small difference from GPT-4o. For efficiency, GPT-4o was used in subsequent tests.

The impact of OSM graph features in the Path Weighter, described in Sec. III-B.2, was examined by comparing cases 7 and 8. When OSM graph features were provided, the F1 score improved by 45.92%. Without this information, the LLM relied on general language knowledge and sometimes produced incorrect weight functions. For example, when given the instruction "avoid slopes," the correct OSM key is `incline`. Without explicit feature data, the LLM may instead generate a non-functional key such as `slope`, leading to failure in applying the intended constraint during graph search. Similar issues were observed for other instructions. The instruction "avoid rain" was sometimes mapped to `building_tunnel` instead of `building_passage`, and "avoid stairs" was occasionally associated with `stairs` rather than `steps`. These errors were resolved when OSM graph features were provided to the Path Weighter.

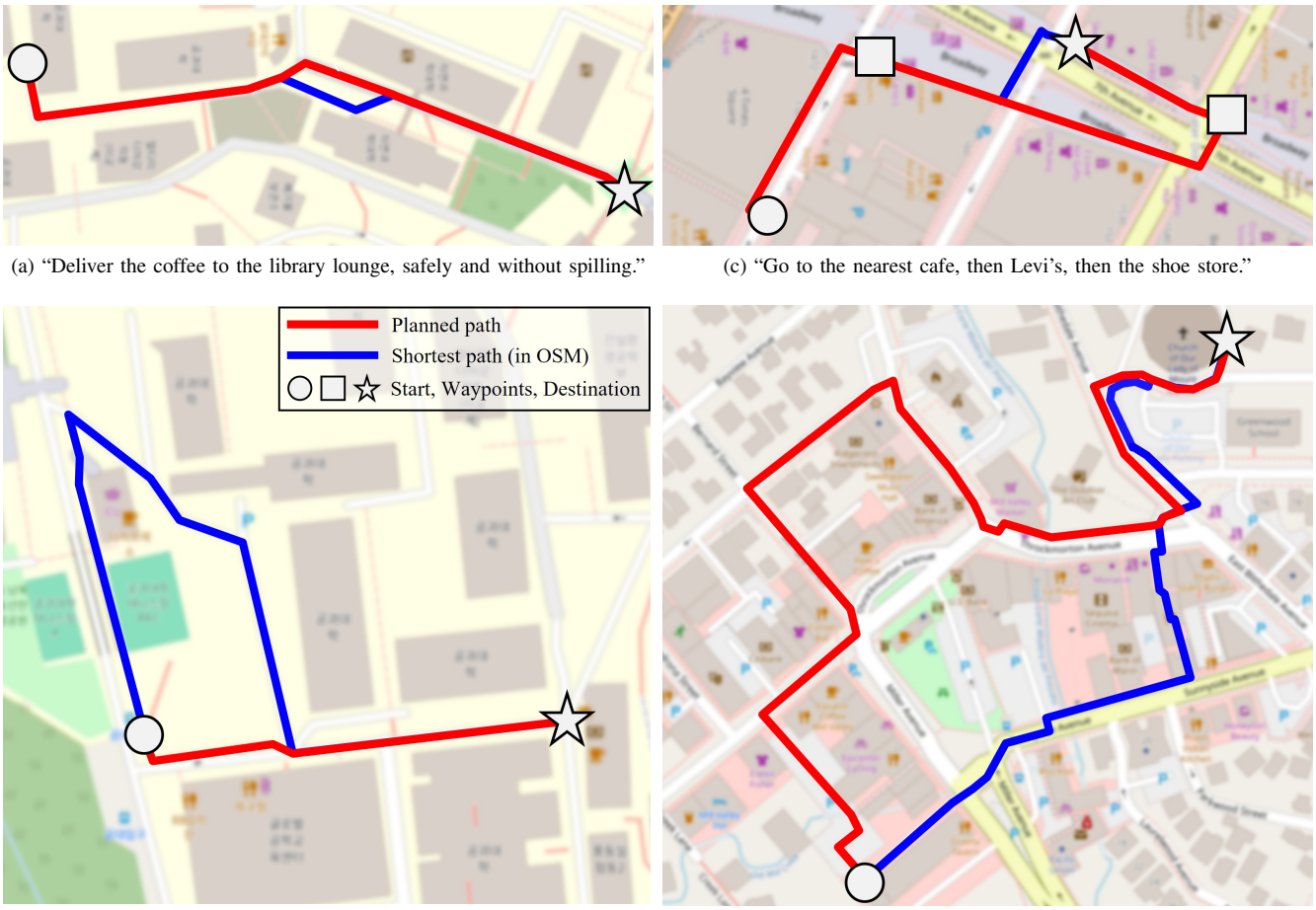
Table IV evaluates the BD Path Planner across four diverse urban and suburban regions, covering a total of 36 scenarios. Using the best configuration consisting of GPT-4 as the LLM, GPT-4o as the VLM, and the inclusion of OSM graph features, the planner achieved F1 scores above 90% in all regions. These results demonstrate the robustness and generalizability of the proposed approach across real-world environments and confirm A3.

Table V reports the number of loaded OSM graph nodes and the corresponding per-module and total processing time of BD Path Planner as the loading radius δ increases from 100 to 500 and 1,000 meters, centered at Times Square, New York. Because the loaded area scales proportionally to δ^2, the area increases by factors of 25 and 100 relative to the 100 meter case. However, the number of nodes increases by factors of 11.75 and 50.82, reflecting the non-uniform spatial distribution of OSM nodes. Overall processing time increases with graph size due to the larger data volume. The Intent Classifier m1, Path Weighter m2, and Path Generator m3 exhibit increasing latency as the number of nodes grows. The LLM-based modules m1 and m2 show approximately linear growth, whereas the A* based Path Generator m3 shows a steeper increase. In contrast, the Path Evaluator m4 remains relatively stable, as it operates on a fixed-size rendered image of the graph and is therefore largely independent of node count. Despite this growth, the total latency remains under 30 seconds even at a 1 kilometer radius. Since planning is executed only once before deployment, this confirms practical real-world feasibility and validates A4.

2) Navigation

We conducted real-world quadruped navigation experiments to validate A4. Initialized by GPS, the robot followed an OSM-based graph path generated from user instructions by the BD Path Planner and executed by the BD Navigator. Results are shown in Fig. 8, with two representative scenarios evaluated.

The first scenario was based on the instruction "Deliver this coffee to my friend in the lounge at the central library.



(a) "Deliver the coffee to the library lounge, safely and without spilling."

(c) "Go to the nearest cafe, then Levi's, then the shoe store."

(b) "Go to the Bank using the fastest route."

(d) "Avoid main roads and take residential streets to the church."

Fig. 6. Visualization of the generated graph paths. The left column presents results from the campus environment for (a) and (b), while the right column shows results from urban and suburban environments for (c) and (d), respectively.

You may go slowly, but please avoid dangerous areas such as stairs, slopes, or construction zones, and take the safest route possible." As shown in Fig. 8(a), the generated path was approximately 330 m long and avoided the staircase shown in Fig. 6(a), instead selecting a longer but safer route over flat terrain. The robot navigated stably, with minor deviations caused by SLAM artifacts and temporary obstacles that occasionally led to local detours. In this scenario, the time required to reach the goal was 406 s, the ground-truth path length was 386.24 m, and the executed trajectory length was 388.48 m, resulting in a path-length inflation of 0.01 and an average travel speed of 1.04 m/s. The mean cross-track error was 1.58 m, with an RMSE of 1.82 m, a maximum error of 5.38 m, and a 95th percentile error of 2.85 m. These deviations were mainly attributed to dynamic obstacles such as pedestrians that required local detours during execution.

The second scenario was based on the instruction "Deliver this document to the staff at the bank. It is acceptable to take a longer route, but please prioritize paths with overhead shelter to avoid the rain." As shown in Fig. 8(b), the generated path was approximately 360 m long and included two building tunnels and four 90-degree turns, reflecting the preference for sheltered routes. Compared to the first scenario, the smoother marble-paved surfaces reduced body shaking and SLAM-related mapping errors. As a result, the robot closely followed the OSM graph path and reached the destination stably. The time required to reach the goal was 355 s, with an executed path length of 361.03 m compared to a ground-truth length of 342.99 m, yielding a path-length inflation of 0.05 and an average travel speed of 1.02 m/s. The mean cross-track error was 0.85 m, with an RMSE of 1.14 m, a maximum error of 2.86 m, and a 95th percentile error of 2.39 m. These errors were smaller than those in the first scenario, primarily due to fewer dynamic obstacles and reduced high-curvature segments, which improved heading estimation under SLAM as shown in Fig. 8.

V. Conclusion

In this paper, we proposed BD-Nav, a framework consisting of the BD Path Planner and the BD Navigator. The planner integrates user instructions with OSM data using LLM- and VLM-based modules to generate intent-aligned graph paths, while the navigator executes these routes on a quadruped robot using SLAM-based localization and rule-based control. Real-world experiments demonstrated the practicality of the proposed approach.

A remaining limitation is that the Path Evaluator relies

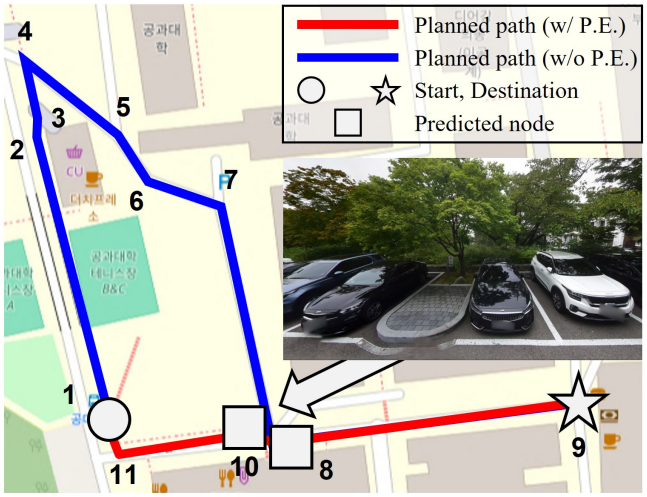
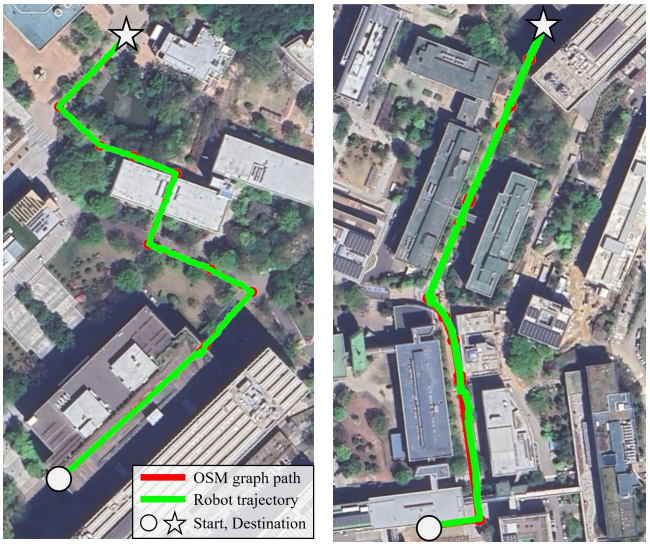


Fig. 7. Example of an incorrect prediction by the Path Evaluator (P.E.).



(a) Scenario 1. (b) Scenario 2.

Fig. 8. Actual robot trajectories over generated graph paths.

on commonsense reasoning rather than direct perceptual feedback, which may occasionally result in incorrect route inferences. Future work will enhance the evaluator to dynamically update graph paths based on user instructions and real-world conditions, and develop an active navigation strategy that leverages perception and exploration when OSM data do not reflect actual traversability.

References

[1] M. Misaros, O.-P. Stan, I.-C. Donca, and L.-C. Miclea, "Autonomous robots for services—state of the art, challenges, and research areas," *Sensors*, vol. 23, no. 10, p. 4962, 2023.

[2] D. Chen and R. Mooney, "Learning to interpret natural language navigation instructions from observations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 25, no. 1, 2011, pp. 859–865.

[3] J. Guerreiro, D. Sato, S. Asakawa, H. Dong, K. M. Kitani, and C. Asakawa, "Cabot: Designing and evaluating an autonomous navigation robot for blind people," in *Proceedings of the 21st international ACM SIGACCESS conference on computers and accessibility*, 2019, pp. 68–82.

[4] S.-W. Kim, G.-P. Gwon, W.-S. Hur, D. Hyeon, D.-Y. Kim, S.-H. Kim, D.-K. Kye, S.-H. Lee, S. Lee, M.-O. Shin *et al.*, "Autonomous campus mobility services using driverless taxi," *IEEE Transactions on intelligent transportation systems*, vol. 18, no. 12, pp. 3513–3526, 2017.

[5] T. Kollar, S. Tellex, D. Roy, and N. Roy, "Toward understanding natural language directions," in *2010 5th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. IEEE, 2010, pp. 259–266.

[6] P. Forte, H. Gupta, H. Andreasson, U. Köckemann, and A. J. Lilienthal, "On robust context-aware navigation for autonomous ground vehicles," *IEEE Robotics and Automation Letters*, 2024.

[7] S. Woo and S.-W. Kim, "Context-based visual-language place recognition," *arXiv preprint arXiv:2410.19341*, 2024.

[8] D. Jung, K. Kim, and S.-W. Kim, "Gotpr: General outdoor text-based place recognition using scene graph retrieval with openstreetmap," *IEEE Robotics and Automation Letters*, 2025.

[9] J. Bao and R. Yonetani, "Path planning using instruction-guided probabilistic roadmaps," *arXiv preprint arXiv:2502.16515*, 2025.

[10] S. C. Marcelyn, Y. Gao, Y. Zhang, X. Gao, and G. Chen, "Pathgpt: Leveraging large language models for personalized route generation," *arXiv preprint arXiv:2504.05846*, 2025.

[11] M. Kuribayashi, K. Uehara, A. Wang, S. Morishima, and C. Asakawa, "Wanderguide: Indoor map-less robotic guide for exploration by blind people," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–21.

[12] C. Kim, K. Kim, M. Oh, H. Baek, J. Lee, D. Jung, S. Woo, Y. Woo, J. Tucker, R. Firoozi *et al.*, "E2map: Experience-and-emotion map for self-reflective robot navigation with language models," *arXiv preprint arXiv:2409.10027*, 2024.

[13] D. Shah, B. Osiński, S. Levine *et al.*, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Conference on robot learning*. PMLR, 2023, pp. 492–504.

[14] J. Wang, D. Huo, Z. Xu, Y. Shi, Y. Yan, Y. Wang, C. Gao, Y. Qiao, and G. Zhou, "Openbench: A new benchmark and baseline for semantic navigation in smart logistics," *arXiv preprint arXiv:2502.09238*, 2025.

[15] W. Burgard, A. B. Cremers, D. Fox, D. Hähnel, G. Lakemeyer, D. Schulz, W. Steiner, and S. Thrun, "Experiences with an interactive museum tour-guide robot," *Artificial intelligence*, vol. 114, no. 1-2, pp. 3–55, 1999.

[16] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The marathon 2: A navigation system," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 2718–2725.

[17] D. Shah, B. Eysenbach, G. Kahn, N. Rhinehart, and S. Levine, "Ving: Learning open-world navigation with visual goals," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 215–13 222.

[18] S. Liu, A. Hasan, K. Hong, R. Wang, P. Chang, Z. Mizrachi, J. Lin, D. L. McPherson, W. A. Rogers, and K. Driggs-Campbell, "Dragon: A dialogue-based robot for assistive navigation with visual language grounding," *IEEE Robotics and Automation Letters*, vol. 9, no. 4, pp. 3712–3719, 2024.

[19] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[20] C. E. Garcia, D. M. Prett, and M. Morari, "Model predictive control: Theory and practice—a survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.

[21] X. Lyu, Q. Han, and B. de Vries, "Procedural modeling of urban layout: population, land use, and road network," *Transportation research procedia*, vol. 25, pp. 3333–3342, 2017.

[22] H. C. Joksch, "The shortest route problem with constraints," *Journal of Mathematical analysis and applications*, vol. 14, no. 2, pp. 191–197, 1966.

[23] L. D. P. Pugliese and F. Guerriero, "A survey of resource constrained shortest path problems: Exact solution approaches," *Networks*, vol. 62, no. 3, pp. 183–200, 2013.

[24] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[25] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality,

long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[26] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.